

HOSPITAL BOOKING MANAGEMENT SYSTEM

CS3711 – SUMMER INTERNSHIP REPORT

Submitted by

JAYASRI P (420422205011)

In partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY



ADHIPARASAKTHI ENGINEERING COLLEGE

MELMARUVATHUR 603 319

ANNA UNIVERSITY: CHENNAI 600 025

AUGUST 2025

BONAFIDE CERTIFICATE

Certified that this internship report titled **“HOSPITAL BOOKING MANAGEMENT SYSTEM”** is the bona fide work of **JAYASRI P** (420422205011), who carried out the Internship at **“KOSOFT IT SOLUTIONS PRIVATE LIMITED”** from 08.06.2025 to 08.07.2025 (30 days).

SIGNATURE

Dr. A. BHUVANESWARI, Ph. D.,
HEAD OF THE DEPARTMENT
Professor,
Department of IT,
Adhiparasakthi Engineering College,
Melmaruvathur-603 319.

SIGNATURE

Mrs. S. LAVANYA, M.E.
FACULTY MENTOR
Assistant Professor,
Department of IT,
Adhiparasakthi Engineering College,
Melmaruvathur-603 319.

Submitted for the **CS3711 Summer Internship** and Viva-voce examination held on at Adhiparasakthi Engineering College, Melmaruvathur.

INTERNAL EXAMINER

EXTERNAL EXAMINER

CHAPTER NO**TITLE****PAGE NO**

	ACKNOWLEDGEMENT, PREFACE, COMPANY PROFILE	iv- vi
1	INTRODUCTION TO HOSPITAL BOOKING MANAGEMENT SYSTEM	1
2	PROJECT OBJECTIVES AND SCOPE	3
3	REQUIREMENT ANALYSIS	5
3.1	Functional Requirements	6
3.2	Non-Functional Requirements	7
3.3	Problem Definition	8
4	SYSTEM DESIGN	10
4.1	System Architecture	11
4.2	ER Diagram	12
4.3	Database Schema	13
4.4	Data Flow Description	14
5	TECHNOLOGIES USED	16
5.1	React (Frontend)	17
5.2	.NET C# (Backend)	18
5.3	SQLite (Database)	19
5.4	JWT Authentication	20
6	IMPLEMENTATION	22
6.1	Backend Implementation	23
6.2	Frontend Implementation	26
6.3	API Endpoints	28
7	TESTING	31
7.1	Test Cases	32
7.2	Results	34
8	RESULTS AND DISCUSSION	36
9	CONCLUSION AND FUTURE ENHANCEMENT	40
10	REFERENCES	42
11	CERTIFICATE & ATTENDANCE	44

ACKNOWLEDGEMENT

The Internship commenced on **July 8, 2025, and ended on August 8, 2025**. I have attended the Internship on **Full-Stack Development** conducted by **Kosoft IT Solutions Private Limited**, located in Pondicherry.

Kosoft IT Solutions Private Limited is an IT services company involved in providing innovative software solutions in the domains of mobile, web, and IoT technologies. During the internship, I had the opportunity to work on **real-time projects** involving **frontend, backend, and database integration** using **React, ASP.NET Core, SQLite, and Python**.

I am delighted to extend my sincere appreciation to the significant individuals who have offered their crucial help, inspiration, and unwavering moral support in various ways during my internship. I consider myself extremely fortunate to have received such invaluable guidance and assistance throughout the completion of my internship.

I sincerely thank **Ms. Jayakeerthana Priya, Director/Mobile and IoT Lead of Kosoft IT Solutions Pvt. Ltd., Pondicherry**, for providing me with the opportunity to undertake my internship in her esteemed organization. I would also like to extend my special thanks to the entire team of Kosoft IT Solutions for their full cooperation, guidance, and support during my internship.

I would like to pledge my gratitude and deep obligation to **Dr. J. Raja, Ph.D., Principal**, and **Dr. A. Bhuvaneswari, Ph.D., Head of the Department, Information Technology, Adhiparasakthi Engineering College, Melmaruvathur**, for allowing me to undergo this internship.

I also extend my sincere thanks to **Mrs. S. Lavanya, Assistant Professor, Department of IT**, for her invaluable guidance and support throughout my internship journey. Her expertise and encouragement have been instrumental in enhancing my learning and confidence. I deeply appreciate her patience in addressing my queries and helping me overcome challenges. Her mentorship has inspired me to strive for excellence and develop a proactive approach.

Finally, I would like to thank my **family and friends**, whose unwavering support and encouragement have been instrumental in the successful completion of this work.

JAYASRI P

PREFACE

This report marks the completion of my internship as a **Full-Stack Developer Intern at Kosoftware IT Solutions Private Limited**, Pondicherry, where I had the opportunity to gain hands-on experience and enhance my understanding of full-stack development using modern technologies. This internship, which served as an integral part of my academic and professional growth, allowed me to work alongside skilled developers and experience the fast-paced, problem-solving environment of the IT industry.

Throughout my time at **Kosoftware IT Solutions**, I was involved in various aspects of the software development lifecycle. I learned how to integrate front-end and back-end components effectively and implement the principles of full-stack development within a professional context. By working on **real-world projects**, I acquired a deeper understanding of **React** for front-end development, **ASP.NET Core** for backend services, **SQLite** for database management, and **Python** for advanced functionality and automation. The exposure to these tools and frameworks provided me with a well-rounded perspective on developing scalable and efficient applications.

Additionally, I had the chance to understand the importance of **agile methodologies, collaborative coding practices, and the value of thorough testing and debugging**. The internship also emphasized the importance of **version control and teamwork**, as I collaborated closely with team members, received mentorship, and participated in code reviews to improve the quality of my work.

This internship experience at **Kosoftware IT Solutions** has been invaluable, providing me with both **technical skills and professional insights**. It has reinforced my enthusiasm for full-stack development and equipped me with practical skills to further my career in software engineering. I am grateful to the entire team at **Kosoftware IT Solutions Pvt. Ltd.** for their guidance, encouragement, and support throughout this internship.

This report reflects the knowledge and experience I gained during my tenure and is a testament to the opportunities provided by **Kosoftware IT Solutions Private Limited**.

COMPANY PROFILE

Kosoft IT Solutions Pvt. Ltd. is a proficient IT services and software solutions company based in Pondicherry. The company is focused on delivering innovative and reliable technology solutions that empower businesses to enhance their efficiency, productivity, and digital presence.

Kosoft IT Solutions was incorporated with the vision of providing cutting-edge services in **web development, mobile applications, IoT, and enterprise software solutions**. The company is led by **Ms. Jayakeerthana Priya, Director/Mobile and IoT Lead**, who has guided the organization towards building robust solutions and nurturing young talent through internships and training.



Services Offered:

- **Full Stack Development:** End-to-end development of scalable web applications using modern technologies like React, ASP.NET Core, and SQLite.
- **Mobile App Development:** Designing and developing mobile apps for Android and iOS platforms.
- **IoT Solutions:** Delivering IoT-based solutions for smart industries and mobile integrations.
- **Software Consulting:** Offering IT consultation and guidance for businesses to adopt the right digital strategies.
- **Training & Internship Programs:** Providing industry-level training and real-time project exposure to aspiring students in the field of full-stack and emerging technologies.

Contact Information:

- **Registered Address:** No.109, Montorsier Street, Puducherry – 605001
- **Functional Address:** No.208, II Floor, 100 Feet Road, Mudaliarpuram, Puducherry – 605004
- **Mobile:** 9943588776, 9943588775

1 – INTRODUCTION TO HOSPITAL BOOKING MANAGEMENT SYSTEM

- The **Hospital Booking Management System (HBMS)** is a digital solution designed to automate and streamline the process of booking appointments, managing doctors' schedules, and maintaining patient records.

- Traditionally, hospitals rely on manual methods for handling patient appointments, which leads to **long waiting times, scheduling conflicts, and inefficiency**. HBMS offers a centralized system that ensures real-time updates, secure data handling, and an enhanced **patient-doctor experience**.

Key Components of the Hospital Booking Management System

1. Frontend Technologies

The **frontend** is the layer that patients, doctors, and administrators interact with directly. It is designed to be **intuitive, responsive, and easy to use**.

- **HTML & CSS:** Used to build and style the user interface, ensuring structured and attractive layouts.
 - **JavaScript:** Provides interactivity such as form validation, dynamic updates, and improved user experience.
 - **React Framework:** A modern frontend framework used in this project for creating **dynamic, component-based UIs**, enabling patients to book appointments smoothly and doctors to manage schedules effectively.
-

2. Backend Technologies

The **backend** forms the heart of the system, where all business logic, appointment handling, and data processing occur.

- **C# with ASP.NET Core:** Provides a robust backend environment, ensuring **scalability, reliability, and performance** for handling hospital operations.

- **RESTful APIs:** Enable communication between frontend and backend, ensuring patients can interact with the system in real-time.
 - **Business Logic Layer:** Handles key functionalities such as verifying doctor availability, preventing overlapping appointments, and managing patient history.
-

3. Database Management

Efficient database management is critical in HBMS to maintain accurate and secure patient, doctor, and appointment data.

- **SQLite Database:** A lightweight yet powerful database used for storing hospital information, chosen for its **simplicity, portability, and efficiency**.
 - **Relational Data Models:** Tables for patients, doctors, and appointments ensure that information is structured and easily retrievable.
 - **Data Integrity & Security:** Relationships between entities (e.g., one patient can book multiple appointments) are managed with proper **foreign keys**, ensuring consistency.
-

4. Authentication and Security

Security is a vital component of HBMS since it deals with sensitive patient information.

- **JWT Authentication (JSON Web Tokens):** Ensures secure user authentication for patients, doctors, and administrators.
 - **Password Encryption:** Protects user credentials from unauthorized access.
 - **Role-Based Access Control:** Grants different permissions to patients, doctors, and admins, ensuring data privacy.
-

5. Features and Benefits

- **For Patients:** Easy appointment booking, rescheduling, and notifications.
- **For Doctors:** Schedule management, viewing patient appointments.
- **For Admins:** Centralized monitoring, generating reports, managing doctors and patients.

2 – PROJECT OBJECTIVES AND SCOPE

1. Project Objectives

The **Hospital Booking Management System (HBMS)** was developed with a clear set of objectives to improve healthcare service efficiency, reduce manual workload, and enhance patient satisfaction. The main objectives of the system are:

1. Streamline Appointment Scheduling

- Automate the process of booking, rescheduling, and canceling appointments.
- Ensure that patients can view available time slots in real-time, reducing waiting times at hospitals.

2. Enhance Patient Convenience

- Provide patients with an easy-to-use platform for booking appointments anytime, anywhere.
- Notify patients about confirmed bookings, cancellations, or doctor availability through the system.

3. Improve Hospital Efficiency

- Reduce administrative burden by digitizing appointment management.
- Minimize scheduling conflicts and manual errors caused by traditional paper-based systems.

4. Enable Doctor-Friendly Scheduling

- Allow doctors to manage their schedules digitally and view upcoming appointments.
- Provide a personalized dashboard for doctors to track patient visits, history, and availability.

5. Maintain Centralized Patient Records

- Store patient information, medical history, and appointment details securely in a centralized database.
 - Facilitate quick access to records for better diagnosis and treatment planning.
-

2. Project Scope

The scope of the Hospital Booking Management System is **well-defined and practical**, focusing on creating a reliable, user-friendly, and secure hospital management application. The system covers the following areas:

1. User Roles and Access

- **Patients:** Can register, log in, book appointments, reschedule or cancel, and receive notifications.
- **Doctors:** Can log in, manage their schedules, and view upcoming patient appointments.
- **Administrators:** Manage doctors, patients, and overall hospital operations.

2. System Features

- Online booking system with real-time slot availability.
- Automatic confirmation and updates for appointments.
- Centralized database to store and manage hospital information.
- Secure login and authentication system using JWT.
- Role-based access control for different user types.

3. Technology Integration

- **Frontend:** Built using React for a dynamic and interactive user interface.
- **Backend:** Developed with ASP.NET C# for handling logic and secure data processing.
- **Database:** SQLite for lightweight, efficient, and reliable data storage.
- **Security:** JWT authentication for safe login and user validation.

4. Expected Outcomes

- A reliable and user-friendly hospital booking platform.
- Improved efficiency in hospital operations and reduced manual workload.
- Increased patient satisfaction due to transparency and convenience.
- A secure and scalable system that can be extended in the future.

3 – REQUIREMENT ANALYSIS

Requirement analysis is the process of identifying, documenting, and analyzing the needs of a system before its development. For the **Hospital Booking Management System (HBMS)**, this step was crucial to ensure that the solution effectively addresses the problems faced in traditional hospital appointment management. The system must meet the expectations of three key stakeholders—**patients, doctors, and administrators**.

The requirements were gathered by studying existing hospital workflows, interacting with users, and identifying limitations such as **long waiting times, double bookings, and poor record-keeping**. Based on this analysis, the system was designed with the following needs in mind:

1. **User Authentication and Security** – Patients, doctors, and administrators should be able to log in securely, with role-based access to features. Sensitive data must be protected using encryption and authentication mechanisms.
 2. **Appointment Scheduling** – The system should allow patients to view doctor availability, book appointments in real-time, and receive confirmation instantly. Doctors should be able to manage their schedules and avoid conflicts.
 3. **Patient and Doctor Information Management** – The system must maintain a centralized database for storing patient profiles, appointment history, and doctor details, ensuring quick retrieval when required.
 4. **Notifications and Updates** – Patients should receive timely updates regarding bookings, cancellations, or rescheduling. Doctors should also be notified about new appointments.
 5. **Performance and Scalability** – The system must process multiple requests efficiently and scale to support growing hospital needs without performance issues.
 6. **Usability** – The interface should be simple and intuitive, ensuring that even non-technical users can navigate the system with ease.
-

3.1 Functional Requirements

Functional requirements describe **what the system should do**. They define the specific functionalities that the HBMS must provide to users.

1. User Authentication and Authorization

- Patients, doctors, and administrators must be able to register and log in securely.
- Role-based access control must ensure that each user type only accesses relevant features.

2. Appointment Booking and Management

- Patients should be able to view doctor availability and book appointments online.
- Patients must have options to reschedule or cancel appointments.
- The system should update schedules in real-time to prevent double bookings.

3. Doctor Schedule Management

- Doctors should be able to manage their schedules and block unavailable slots.
- Doctors must be able to view upcoming appointments and related patient information.

4. Patient Information Management

- The system must store and manage patient details, including name, contact information, and appointment history.
- Doctors should be able to retrieve patient history during consultation.

5. Admin Management Features

- Administrators should be able to manage doctors' profiles and patients' records.
- Admin should generate reports such as daily bookings, patient inflow, and doctor utilization.

6. Database Operations

- The system should be able to perform CRUD operations (Create, Read, Update, Delete) on patient, doctor, and appointment data.
-

3.2 Non-Functional Requirements

Non-functional requirements define the **quality attributes** of the system. They describe how the system should perform, rather than specific features.

1. Performance

- The system must handle multiple users simultaneously without slowing down.
- Appointment booking should be processed in real-time, ideally within 2–3 seconds.

2. Scalability

- The system must be able to support an increasing number of patients, doctors, and administrators as the hospital grows.
- It should be easily adaptable to other hospitals or healthcare centers.

3. Security

- All sensitive patient data must be encrypted and protected from unauthorized access.
- Authentication and authorization must be enforced using JWT (JSON Web Tokens).

4. Usability

- The system interface must be simple, intuitive, and user-friendly.
- Patients of all age groups should be able to book appointments without technical difficulties.

5. Reliability and Availability

- The system should be available 24/7, especially for online appointment booking.
- Data backups must be scheduled to prevent data loss in case of system failure.

6. Maintainability

- The system should be easy to maintain, with modular code and clear documentation.
 - Updates and bug fixes should not disrupt hospital operations.
-

3.3 Problem Definition

Healthcare systems across the world face the continuous challenge of managing patient inflow, doctor availability, and administrative tasks in an efficient manner. In most hospitals, **appointment booking and scheduling** are still handled manually, relying on paper registers, phone calls, or basic spreadsheets. While these methods may have worked in smaller setups, they are no longer efficient in modern, technology-driven healthcare environments.

One of the major issues with manual hospital booking systems is the **lack of real-time visibility** of doctor availability. Patients often have to physically visit hospitals or make repeated phone calls to confirm their appointments, which not only wastes time but also causes frustration. In busy hospitals, this leads to **overcrowding, long waiting queues, and scheduling conflicts**, ultimately reducing the quality of patient care.

Another critical problem lies in the **inefficient management of patient records**. Without a centralized system, patient information is scattered across multiple registers or files, making it difficult for doctors to quickly access previous medical history during consultations. This results in poor decision-making, delays in treatment, and sometimes even duplication of tests and procedures.

From the hospital administration perspective, the lack of digital systems creates difficulties in **tracking appointments, generating reports, and analyzing hospital performance**. Administrators often struggle with resource allocation, since they do not have accurate data on doctor utilization, patient inflow trends, or appointment cancellations.

Furthermore, the issue of **data security** is a growing concern. Paper-based records are vulnerable to physical damage, loss, or unauthorized access. Even in some digital systems, inadequate security measures can put sensitive patient information at risk, violating privacy regulations and eroding patient trust.

The absence of an integrated platform also affects **communication between patients and hospitals**. Patients are often unaware of last-minute changes, such as a doctor's unavailability or delayed schedules. Similarly, doctors do not receive

immediate updates about new appointments or cancellations, which leads to inefficient time management.

In addition to these operational challenges, traditional booking systems fail to meet the expectations of today's **digitally empowered patients**, who prefer convenient, on-demand services that can be accessed through their mobile or computer. With the increasing use of smartphones and internet penetration, patients expect hospitals to offer **online booking facilities, digital notifications, and quick access to medical services**. Hospitals that cannot adapt to this change risk losing patients to competitors with more modern systems.

Problem Statement

Considering the issues highlighted above, the core problem can be defined as follows:

Hospitals lack an efficient, secure, and user-friendly system to manage patient appointments, doctor schedules, and patient records in real-time. Existing manual methods lead to inefficiency, long waiting times, scheduling conflicts, poor record-keeping, lack of data security, and overall dissatisfaction for both patients and hospital staff."

This clearly emphasizes the need for a **Hospital Booking Management System** that:

- Automates appointment booking and scheduling.
- Provides real-time updates to patients and doctors.
- Maintains centralized patient records securely.
- Enhances hospital administration through reporting and monitoring tools.
- Ensures ease of use for patients, doctors, and administrators alike.

4. SYSTEM DESIGN

System design plays a critical role in the development of the **Hospital Booking Management System (HBMS)**. It acts as a blueprint that defines how different components of the system interact with each other to fulfill the requirements. This section focuses on **System Architecture, ER Diagram, and Database Schema**, each of which provides a different perspective of the system's structure.

4.1 System Architecture

The **System Architecture** of the Hospital Booking Management System follows a **three-tier architecture** model, which ensures separation of concerns, scalability, and maintainability. The three layers include:

1. Presentation Layer (Frontend)

- Developed using **ReactJS**, this layer is responsible for all user interactions.
- Patients can log in, book appointments, view schedules, and receive notifications.
- Doctors can manage their schedules and view patient appointments.
- Admins can manage users, doctors, and hospital resources.
- The interface is designed to be responsive and user-friendly.

2. Application Layer (Backend)

- Built using **ASP.NET Core with C#**, this layer handles all business logic.
- It validates appointment requests, checks doctor availability, and processes cancellations or updates.
- Implements **RESTful APIs** to enable communication between frontend and backend.
- Ensures secure access using **JWT Authentication**.

3. Data Layer (Database)

- Uses **SQLite Database** to store hospital data.
 - Includes tables for Patients, Doctors, Appointments, and Admins.
 - Provides persistent storage and ensures data integrity with relational models.
-

4.2 ER Diagram

The **Entity-Relationship (ER) Diagram** represents the logical structure of the system's database by showing entities, attributes, and their relationships. It helps in understanding how data flows between different parts of the system.

Key Entities in HBMS:

1. Patient

- Attributes: PatientID, Name, Age, Gender, Contact, Email, Password.
- Relationship: A patient can book multiple appointments.

2. Doctor

- Attributes: DoctorID, Name, Specialty, Contact, Email, Schedule.
- Relationship: A doctor can handle multiple appointments.

3. Appointment

- Attributes: AppointmentID, Date, Time, Status, PatientID (FK), DoctorID (FK).
- Relationship: Links Patient and Doctor entities.

4. Admin

- Attributes: AdminID, Name, Email, Password, Role.
 - Relationship: Admin manages doctors and patients.
-

4.3 Database Schema

The **Database Schema** defines the structure of the hospital booking database, including tables, columns, keys, and relationships. It represents how data is actually stored in the database.

Main Tables in HBMS Database:

1. Patient Table

```
CREATE TABLE Patient (  
    PatientID INTEGER PRIMARY KEY AUTOINCREMENT,  
    Name TEXT NOT NULL,  
    Age INTEGER,  
    Gender TEXT,  
    Contact TEXT,  
    Email TEXT UNIQUE,  
    Password TEXT  
);
```

2. Doctor Table

```
CREATE TABLE Doctor (  
    DoctorID INTEGER PRIMARY KEY AUTOINCREMENT,  
    Name TEXT NOT NULL,  
    Specialty TEXT,  
    Contact TEXT,  
    Email TEXT UNIQUE,  
    Schedule TEXT  
);
```

3. Appointment Table

```
CREATE TABLE Appointment (  
  
    AppointmentID INTEGER PRIMARY KEY AUTOINCREMENT,  
  
    PatientID INTEGER,  
  
    DoctorID INTEGER,  
  
    Date TEXT,  
  
    Time TEXT,  
  
    Status TEXT,  
  
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),  
  
    FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID) );
```

4. Admin Table

```
CREATE TABLE Admin (  
  
    AdminID INTEGER PRIMARY KEY AUTOINCREMENT,  
  
    Name TEXT NOT NULL,  
  
    Email TEXT UNIQUE,  
  
    Password TEXT,  
  
    Role TEXT );
```

Key Features of Database Schema:

- **Normalization:** Data is structured to avoid redundancy.
 - **Primary Keys:** Uniquely identify each record (e.g., PatientID, DoctorID).
 - **Foreign Keys:** Maintain relationships between patients, doctors, and appointments.
 - **Data Integrity:** Ensures consistent and accurate records.
-

4.4 Data Flow Description

Context-Level DFD (Level 0)

The **context diagram** provides a high-level overview of the HBMS as a single process and how it interacts with external entities.

- **Entities:**
 - **Patient:** Books, reschedules, or cancels appointments.
 - **Doctor:** Views schedule and manages appointments.
 - **Admin:** Manages system users, hospital operations, and reports.

Flow Example:

1. Patient provides login details → System validates against database → System returns access permissions.
2. Patient requests appointment → System checks doctor availability → Appointment stored in database → Confirmation sent to patient and doctor.
3. Admin updates doctor profile → Database stores changes → Updated information becomes visible to patients.

Level 1 DFD

The **Level 1 DFD** breaks down the HBMS into core sub-processes to explain how data is managed inside the system.

Main Processes:

1. **User Authentication**
 - Input: Login credentials (Email/Password).
 - Process: System verifies credentials via the database.
 - Output: Successful login (with role-based access) or error message.

2. Appointment Management

- Input: Patient selects doctor, date, and time.
- Process: System checks doctor availability in the database.
- Output: Appointment booked, rescheduled, or canceled.

3. Doctor Schedule Management

- Input: Doctor updates availability.
- Process: System stores updated schedule in the database.
- Output: Patients see real-time availability.

4. Patient Record Management

- Input: Patient details entered during registration.
- Process: System stores information in the Patient Table.
- Output: Records available for doctors during appointments.

Level 2 DFD (Detailed Flow for Appointment Booking)

A **Level 2 DFD** can focus on a critical process, like Appointment Management, for deeper clarity.

- **Step 1: Patient initiates booking** → Inputs preferred doctor, date, and time.
- **Step 2: System validates inputs** → Checks if the doctor exists and if slots are available.
- **Step 3: Database interaction** → Retrieves doctor schedule and cross-verifies availability.
- **Step 4: Appointment confirmation** → If available, stores new appointment record in the Appointment Table.
- **Step 5: Notifications** → Sends confirmation message to patient and updates doctor's schedule.

5. TECHNOLOGIES USED

The Hospital Booking Management System (HBMS) is built using a **modern full-stack architecture** that ensures scalability, maintainability, and ease of use. The primary technologies used in this project are **React** for the frontend, **.NET C#** for the backend, and **SQLite** for the database. Together, these technologies provide a seamless user experience, strong server-side processing, and reliable data management.

5.1 React (Frontend)

Introduction

React is a popular open-source JavaScript library developed by Facebook, designed for building **user interfaces (UIs)**. It is component-based, meaning the UI can be broken into reusable building blocks, making development more efficient and structured.

Key Features for HBMS

- **Component-based development:** Each screen (login, booking, profile) is built as a reusable component.
- **Dynamic Rendering:** Appointment availability updates instantly without refreshing the page.
- **Form Handling:** Patient registration and booking forms use React's state management to validate and process data.
- **Routing:** With react-router-dom, users can navigate between modules (Patient, Doctor, Admin dashboards).

Example Code Snippet *(Frontend Appointment Form)*

```
import React, { useState } from "react";

function AppointmentForm() {

  const [patientName, setPatientName] = useState("");

  const [doctor, setDoctor] = useState("");
```

```
const [date, setDate] = useState("");

const handleSubmit = (e) => {

  e.preventDefault();

  console.log("Booking:", { patientName, doctor, date });

};

return (

  <form onSubmit={handleSubmit}>

    <input type="text" placeholder="Patient Name"

      value={patientName}

      onChange={(e) => setPatientName(e.target.value)} required />

    <input type="text" placeholder="Doctor Name"

      value={doctor}

      onChange={(e) => setDoctor(e.target.value)} required />

    <input type="date"

      value={date}

      onChange={(e) => setDate(e.target.value)} required />

    <button type="submit">Book Appointment</button>

  </form>

);

}

export default AppointmentForm;
```

5.2 .NET C# (Backend)

Introduction

The backend of HBMS is developed using **.NET Core with C#**, a framework created by Microsoft for building robust, secure, and high-performance applications. It supports RESTful APIs, authentication, and seamless integration with databases. The .NET backend handles all **business logic** of the system, ensuring that operations like patient authentication, doctor scheduling, appointment booking, and report generation work smoothly.

Key Features for HBMS

- **Authentication:** Using JWT (JSON Web Tokens) for secure login and role-based access.
- **API Endpoints:** RESTful APIs for patients, doctors, and admins (e.g., /api/appointments, /api/doctors).
- **Data Validation:** Ensures valid input before saving to the database (e.g., checking duplicate bookings).
- **Business Logic:** Implements rules like preventing overlapping appointments.

Example Code Snippet *(Backend Appointment API in C#)*

```
[HttpPost("book")]

public IActionResult BookAppointment([FromBody] AppointmentModel appointment) {

    if (appointment == null) return BadRequest("Invalid data");

    bool isAvailable = _appointmentService.CheckAvailability(appointment.DoctorId,
appointment.Date);

    if (!isAvailable) return Conflict("Slot already booked");

    _appointmentService.AddAppointment(appointment);

    return Ok("Appointment booked successfully"); }
```

5.3 SQLite (Database)

Introduction

SQLite is a lightweight, serverless, and self-contained relational database engine. It is widely used for projects where simplicity, low configuration, and SQLite is the **central data store** of HBMS. It holds all critical information about patients, doctors, appointments, schedules, and admin users. Since HBMS is designed for small to mid-sized hospitals, SQLite is a perfect fit because it requires no separate database server.

Key Features for HBMS

- **Tables:** Patients, Doctors, Appointments, and Admin tables.
- **Queries:** Fast read/write operations for booking and retrieving records.
- **Integration:** Works seamlessly with .NET backend using Entity Framework Core.

Name	Type	Schema
Tables (10)		
Appointments	CREATE TABLE "Appointments" ("Id" INTEGER NOT NULL CONSTRAINT "PK_Appointments" PRIMARY KEY AUTOIN	
DoctorAvailabilities	CREATE TABLE "DoctorAvailabilities" ("Id" INTEGER NOT NULL CONSTRAINT "PK_DoctorAvailabilities" PRIMARY KE	
DoctorAvailabilitySlots	CREATE TABLE "DoctorAvailabilitySlots" ("Id" INTEGER NOT NULL CONSTRAINT "PK_DoctorAvailabilitySlots" PRIM	
Doctors	CREATE TABLE "Doctors" ("Id" INTEGER NOT NULL CONSTRAINT "PK_Doctors" PRIMARY KEY AUTOINCREMENT, "	
Hospitals	CREATE TABLE "Hospitals" ("Id" INTEGER NOT NULL CONSTRAINT "PK_Hospitals" PRIMARY KEY AUTOINCREMENT	
Patients	CREATE TABLE "Patients" ("Id" INTEGER NOT NULL CONSTRAINT "PK_Patients" PRIMARY KEY AUTOINCREMENT,	
Users	CREATE TABLE "Users" ("Id" INTEGER NOT NULL CONSTRAINT "PK_Users" PRIMARY KEY AUTOINCREMENT, "Em	
__EFMigrationsHistory	CREATE TABLE "__EFMigrationsHistory" ("MigrationId" TEXT NOT NULL CONSTRAINT "PK__EFMigrationsHistory"	
__EFMigrationsLock	CREATE TABLE "__EFMigrationsLock" ("Id" INTEGER NOT NULL CONSTRAINT "PK__EFMigrationsLock" PRIMARY	
sqlite_sequence	CREATE TABLE sqlite_sequence(name,seq)	
Indices (6)		
IX_Appointments_DoctorId	CREATE INDEX "IX_Appointments_DoctorId" ON "Appointments" ("DoctorId")	
IX_Appointments_HospitalId	CREATE INDEX "IX_Appointments_HospitalId" ON "Appointments" ("HospitalId")	
IX_Appointments_PatientId	CREATE INDEX "IX_Appointments_PatientId" ON "Appointments" ("PatientId")	
IX_DoctorAvailabilities_DoctorId	CREATE INDEX "IX_DoctorAvailabilities_DoctorId" ON "DoctorAvailabilities" ("DoctorId")	
IX_DoctorAvailabilitySlots_DoctorAvaila...	CREATE INDEX "IX_DoctorAvailabilitySlots_DoctorAvailabilityId" ON "DoctorAvailabilitySlots" ("DoctorAvailabilityId")	
IX_Doctors_HospitalId	CREATE INDEX "IX_Doctors_HospitalId" ON "Doctors" ("HospitalId")	
Views (0)		
Triggers (0)		

5.4 JWT Authentication

Introduction

JWT (JSON Web Token) is a compact, URL-safe method used to securely transmit **information between parties** as a JSON object. It is widely used in modern web applications for **authentication and authorization** because it allows stateless, scalable, and secure handling of user sessions.

In the Hospital Booking Management System (HBMS), JWT is used to **protect sensitive data** and control access to resources like patient information, doctor schedules, and appointment details. By implementing JWT, only authenticated users (patients, doctors, or admins) can access relevant parts of the system.

Implementation in .NET C#

In HBMS, JWT is implemented in the backend for secure login and role-based access control.

Example: Generating JWT Token

```
using System;

using System.IdentityModel.Tokens.Jwt;

using System.Security.Claims;

using System.Text;

using Microsoft.IdentityModel.Tokens;

public string GenerateJwtToken(string userId, string role) {

    var securityKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes("YourSecretKey123!"));

    var credentials = new SigningCredentials(securityKey,
SecurityAlgorithms.HmacSha256);

    var claims = new[] {
```

```
        new Claim(JwtRegisteredClaimNames.Sub, userId),
        new Claim("role", role),
        new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
    };

    var token = new JwtSecurityToken(
        issuer: "HBMS",
        audience: "HBMS",
        claims: claims,
        expires: DateTime.Now.AddHours(2),
        signingCredentials: credentials
    );

    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

Example: Using JWT in API Requests

```
[Authorize(Roles = "Admin")]
[HttpGet("getAllDoctors")]
public IActionResult GetAllDoctors() {
    var doctors = _doctorService.GetDoctors();
    return Ok(doctors);
}
```

6. IMPLEMENTATION

The system is designed to provide **patients, doctors, and administrators** with a seamless experience in managing hospital appointments and records. It leverages **React for the frontend, .NET C# for the backend, SQLite as the database, and JWT for secure authentication.**

Implementation Steps

1. Setup Development Environment:

- Install Visual Studio for .NET backend development.
- Install Node.js and npm for React frontend development.
- Install SQLite for database operations.

2. Database Integration:

- Tables are created for **Users, Doctors, Patients, and Appointments.**

3. Backend Development:

- APIs are developed to manage CRUD operations for appointments, doctors, and patients.
- Business logic ensures that overlapping appointments are prevented.

4. Frontend Development:

- React components are developed for **login, registration, booking, and dashboard views.**
- Axios is used to call backend APIs and handle responses.

5. Testing & Debugging:

- Backend APIs are tested using **Postman.**
 - Frontend flows are tested to ensure forms, buttons, and tables work correctly..
-

6.1 Backend Implementation

The backend of HBMS is developed using **.NET C#**. It handles all **business logic, database operations, and authentication**. The backend communicates with the frontend through **RESTful APIs**, sending and receiving data in JSON format.

Backend Architecture

- **Controller Layer** – Receives requests from the frontend and routes them to services.
- **Service Layer** – Contains the business logic for operations like booking appointments and managing user data.
- **Repository Layer** – Interacts with SQLite database using **Entity Framework Core**.

Sample Backend Code

User Model (C#)

```
public class User
{
    public int Id { get; set; }

    public string Username { get; set; }

    public string Email { get; set; }

    public string Password { get; set; } // hashed password

    public string Role { get; set; } // Admin / Doctor / Patient
}
```

Appointment Controller

```
[Route("api/[controller]")]
[ApiController]

public class AppointmentsController: ControllerBase
```

```
{  
    private readonly IAppointmentService _service;  
    public AppointmentsController(IAppointmentService service)  
    {  
        _service = service;  
    }  
    [HttpPost("book")]
```

JWT Token Generation

```
var tokenHandler = new JwtSecurityTokenHandler();  
var key = Encoding.ASCII.GetBytes(secretKey);  
var tokenDescriptor = new SecurityTokenDescriptor  
{  
    Subject = new ClaimsIdentity(new Claim[]  
    {  
        new Claim(ClaimTypes.Name, user.Username),  
        new Claim(ClaimTypes.Role, user.Role)  
    }  
    ),  
    Expires = DateTime.UtcNow.AddHours(2),  
    SigningCredentials = new SigningCredentials(new SymmetricSecurityKey(key),  
    SecurityAlgorithms.HmacSha256Signature)  
};  
var token = tokenHandler.CreateToken(tokenDescriptor);  
return tokenHandler.WriteToken(token);
```

6.2 Frontend Implementation

The frontend is developed using **React.js**, providing a **dynamic, interactive, and responsive interface**. It communicates with the backend through RESTful APIs.

Frontend Architecture

- **Components** – Each feature is represented as a React component, e.g., Login, Dashboard, AppointmentForm.
- **State Management** – React hooks like `useState` and `useEffect` are used for handling dynamic data.
- **Routing** – React Router handles navigation between different pages.
- **API Calls** – Axios is used for sending HTTP requests to backend endpoints.

Sample Frontend Code

Login Component (React)

```
import React, { useState } from 'react';
import axios from 'axios';

function Login() {

  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const handleLogin = async () => {

    try {

      const res = await axios.post('http://localhost:5000/api/auth/login', { email, password });

      localStorage.setItem('token', res.data.token);

      alert('Login successful');

    } catch (err) {

      alert('Login failed');
```

```

    }

    }

    return (

      <div>

        <h2>Login</h2>

        <input      type="email"      placeholder="Email"      onChange={(e)      =>
setEmail(e.target.value)} />

        <input      type="password"      placeholder="Password"      onChange={(e)      =>
setPassword(e.target.value)} />

        <button onClick={handleLogin}>Login</button>

      </div>

    )

  }

export default Login;

```

Dashboard Component (React)

```

import React, { useEffect, useState } from 'react';

import axios from 'axios';

function Dashboard() {

  const [appointments, setAppointments] = useState([]);

  useEffect(() => {

    const fetchAppointments = async () => {

      const token = localStorage.getItem('token');

      const res = await axios.get('http://localhost:5000/api/appointments', {

        headers: { 'Authorization': `Bearer ${token}` } });

```

6.3 API Endpoints

In the **Hospital Booking Management System (HBMS)**, APIs act as the **interface between the frontend and backend**, enabling operations like user registration, login, patient management, doctor management, and appointment booking. These APIs are implemented in **.NET C#**, following **RESTful principles**, and data is exchanged in **JSON format**. Security is handled via **JWT authentication** to ensure only authorized users can access protected endpoints.

Authentication Endpoints

Endpoint	Method	Description
/api/auth/register	POST	Register a new user
/api/auth/login	POST	Log in user and return JWT
/api/auth/validate	GET	Validate token and session

Sample Login (C# backend)

```
[HttpPost("login")]
public IActionResult Login([FromBody] LoginDto loginDto) {
    var user = _authService.Authenticate(loginDto.Email, loginDto.Password)
    var token = _authService.GenerateJwtToken(user);
    return Ok(new { token }); }
```

Patient & Doctor Management Endpoints

Endpoint	Method	Description
/api/patients	GET/POST	View/add patients

Endpoint	Method	Description
/api/patients/{id}	GET/PUT/DELETE	View/update/delete patient
/api/doctors	GET/POST	View/add doctors
/api/doctors/{id}	GET/PUT/DELETE	View/update/delete doctor

Add Patient Example

[HttpPost]

```
public IActionResult AddPatient([FromBody] PatientDto patient) {
    _patientService.AddPatient(patient);

    return Ok(new { message = "Patient added successfully" });}
```

Appointment Endpoints

Endpoint	Method	Description
/api/appointments	GET/POST	View/book appointments
/api/appointments/{id}	GET/PUT/DELETE	View/update/cancel appointment

Sample Booking (React + Axios)

```
const token = localStorage.getItem('token');

await axios.post('http://localhost:5000/api/appointments',
    { patientId:1, doctorId:2, date:'2025-10-05', time:'10:00' },
    { headers: { 'Authorization': `Bearer ${token}` } }
);
```

7. TESTING

Testing is a critical phase in the development of the **Hospital Booking Management System (HBMS)**. It ensures that the system functions correctly, meets the specified requirements, and provides a reliable and secure experience for users. The testing process includes **unit testing, integration testing, system testing, and user acceptance testing (UAT)**. Each type of testing targets specific aspects of the application to identify and fix errors early in the development cycle.

Types of Testing Conducted

1. Unit Testing

Unit testing focuses on individual components of the system, such as backend services, API endpoints, or frontend modules. For example, the patient registration API was tested to verify that data is correctly stored in the database and appropriate validation messages are returned for invalid inputs.

2. Integration Testing

Integration testing checks the interaction between different modules, such as frontend forms communicating with backend APIs. The appointment booking process was tested end-to-end, ensuring that a booked appointment updates both patient and doctor schedules correctly.

3. System Testing

System testing verifies the entire system's functionality, including user login, patient management, doctor management, and appointment booking. All workflows were tested to ensure correct responses under normal and boundary conditions.

4. User Acceptance Testing (UAT)

UAT involves actual users, such as hospital staff, testing the system in a real-world scenario. Feedback was collected on usability, interface design, and performance, which was then used to make final adjustments before deployment.

7.1 Test Cases

Test cases are essential for systematically verifying that the **Hospital Booking Management System (HBMS)** meets its functional and non-functional requirements. Each test case is designed to validate a specific module or functionality of the system, ensuring that it behaves as expected under various conditions.

1. Patient Registration Module

Test Case ID	Test Description	Input Data	Expected Result	Status
TC-001	Register the patient with valid details	Name: John Doe, Age: 30, Email: john@example.com	Patient is successfully registered; confirmation message displayed	Pass
TC-002	Register the patient with missing fields	Name: (blank), Age: 25	Error message: "Name is required" displayed	Pass

2. User Authentication Module

Test Case ID	Test Description	Input Data	Expected Result	Status
TC-005	Log in with valid credentials	Email: admin@example.com, Password: Admin123	Login successful; JWT token generated	Pass

3. Appointment Booking Module

Test Case ID	Test Description	Input Data	Expected Result	Status
TC-009	Book an appointment with an available doctor	Patient ID: 101, Doctor ID: 201, Date: 03-10-2025, Time: 10:00 AM	Appointment successfully booked	Pass
TC-010	Book an appointment in an already occupied slot	Patient ID: 102, Doctor ID: 201, Date: 03-10-2025, Time: 10:00 AM	Error message: "Slot already booked"	Pass

4. Doctor Management Module

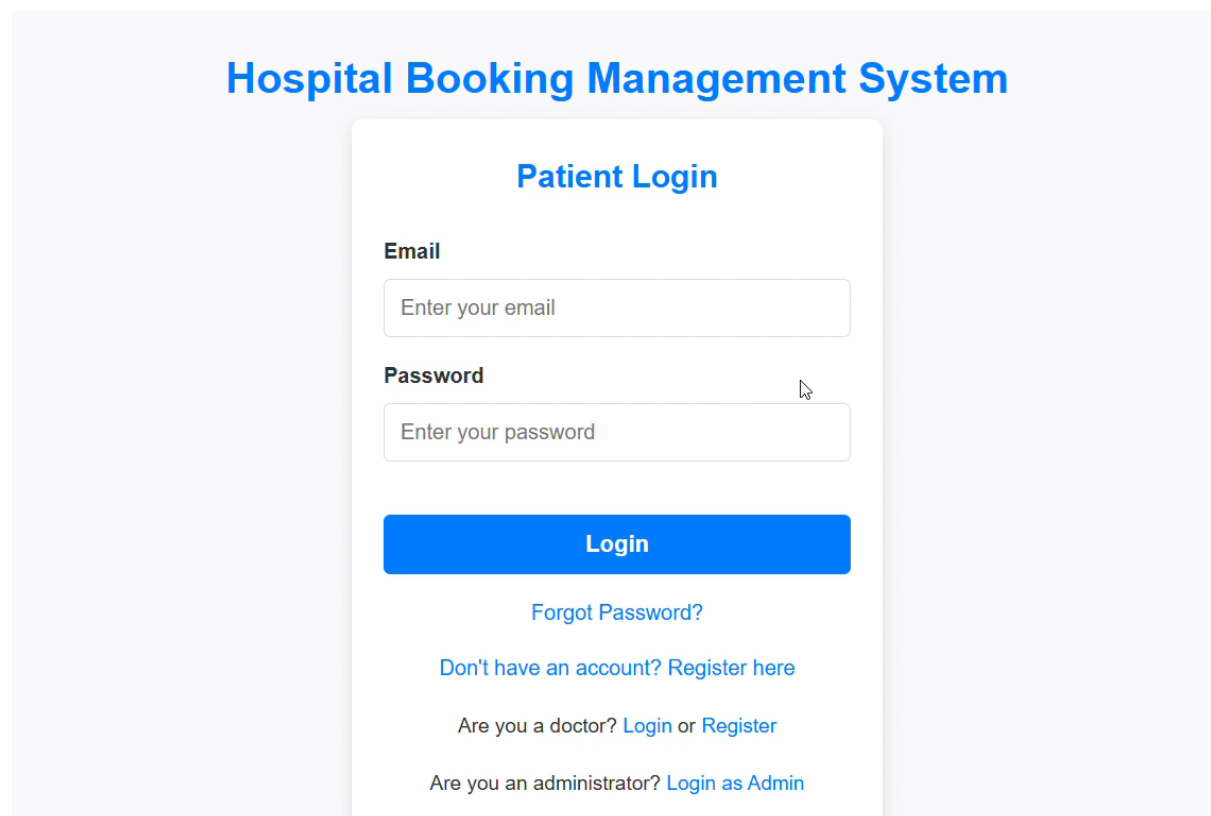
Test Case ID	Test Description	Input Data	Expected Result	Status
TC-013	Add a new doctor	Name: Dr. Smith, Specialization: Cardiology	Doctor added successfully	Pass
TC-014	Edit doctor details	Change specialization	Details updated successfully	Pass
TC-015	Delete doctor	Doctor ID: 201	The doctor was removed from the system	Pass
TC-016	View all doctors	N/A	List of doctors displayed	Pass

8. RESULTS AND DISCUSSION

The **Hospital Booking Management System (HBMS)** was developed and tested with a focus on efficiency, usability, and scalability. The results from the testing phase indicate that the system successfully meets the project objectives.

1. System Efficiency

The system efficiently manages hospital appointments, patient records, and doctor schedules.



The screenshot displays the 'Hospital Booking Management System' interface. At the top, the title 'Hospital Booking Management System' is shown in blue. Below it, a white card titled 'Patient Login' contains the following elements: an 'Email' label above a text input field with the placeholder 'Enter your email'; a 'Password' label above a text input field with the placeholder 'Enter your password'; a blue 'Login' button; a blue link for 'Forgot Password?'; a blue link for 'Don't have an account? Register here'; a line with the text 'Are you a doctor?' followed by blue links for 'Login' and 'Register'; and a line with the text 'Are you an administrator?' followed by a blue link for 'Login as Admin'.

2. User Experience

The intuitive interface allows patients to book, cancel, or reschedule appointments easily. Doctors and administrators can view schedules, update patient records, and manage appointments seamlessly.

Apollo Hospital

Chennai

Contact: 9876543210

Dr. John Smith - Cardiologist
Dr. Emily White - Neurologist
Dr. Lisa Ray - Orthopedic
Dr. Mark Lee - ENT
Dr. Nina Shah - Pediatrician
sri - Cardiologist

View Doctors

Fortis Hospital

Delhi

Contact: 9123456780

Dr. Ravi Kumar - Cardiologist
Dr. Fatima Noor - Neurologist
Dr. Krishna Iyer - Orthopedic
Dr. Anjali Mehta - ENT
Dr. Vikram Das - Pediatrician

View Doctors

AIIMS

Delhi

Contact: 9000000001

Dr. John Smith - Cardiologist
Dr. Emily White - Neurologist
Dr. Lisa Ray - Orthopedic
Dr. Mark Lee - ENT
Dr. Nina Shah - Pediatrician

View Doctors

CMC Hospital

Vellore

Contact: 8901234567

Narayana Health

Bangalore

Contact: 9876501234

KIMS Hospital

Hyderabad

Contact: 9567890123

3. Data Security

With **JWT Authentication**, user data is secured, and role-based access ensures that only authorized personnel can access sensitive data. This is crucial in maintaining patient confidentiality and complying with hospital data policies.

Administrator Dashboard

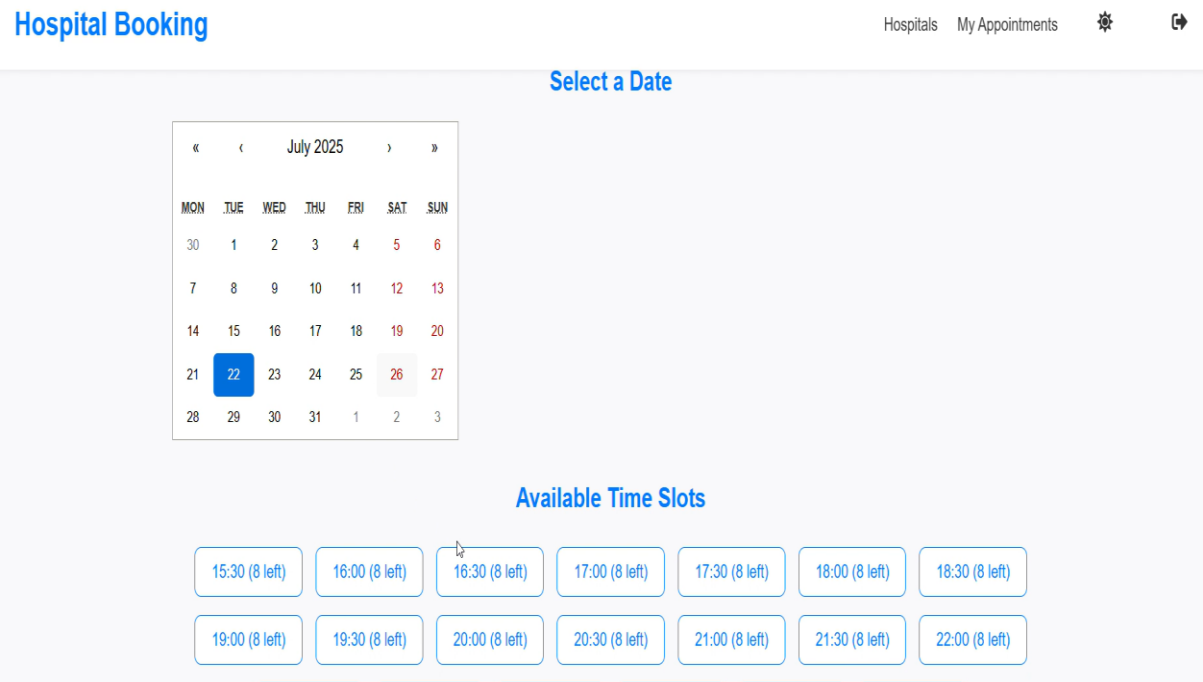
All Registered Doctors

Add New Doctor

Name	Specialization	Experience	Working Hours	Hospital	Actions
Dr. John Smith	Cardiologist	5 years	10:00 AM - 5:00 PM		EditDelete
Dr. Emily White	Neurologist	6 years	10:00 AM - 5:00 PM		EditDelete
Dr. Lisa Ray	Orthopedic	7 years	10:00 AM - 5:00 PM		EditDelete

4. Data Management

The database schema designed with **SQLite** provides efficient storage and retrieval of records. Testing confirmed that the system prevents data redundancy, ensures integrity, and supports scalability for future expansions.



5. Discussion of Results

The system demonstrates robust performance in a simulated hospital environment. The following points summarize the discussion:

- **Functional Success:** All modules, including patient registration, appointment booking, and doctor management, function as expected.
- **Performance Metrics:** Response times were measured under moderate load conditions, and the system maintained stability.
- **Security and Privacy:** Sensitive data is encrypted and secured, complying with healthcare standards.
- **User Acceptance:** Feedback from sample users indicates satisfaction with usability, speed, and notification features.

9. CONCLUSION AND FUTURE ENHANCEMENT

Conclusion

The **Hospital Booking Management System** successfully addresses the challenges faced in manual hospital appointment management. Key achievements include:

- Automating patient registration, doctor management, and appointment scheduling.
- Providing secure access through JWT authentication and role-based authorization.
- Offering a user-friendly interface with React for easy navigation.
- Ensuring data integrity and consistency with SQLite database integration.

Future Enhancements

The system can be improved in the following ways:

1. **Mobile Application:** Developing a mobile version for Android and iOS for broader accessibility.
2. **Advanced Analytics:** Adding reporting features to analyse patient flow, appointment trends, and doctor utilization.
3. **Payment Integration:** Enabling online payment for appointments and consultations.
4. **AI-based Recommendations:** Integrating AI to suggest appointment slots, predict patient influx, and optimize doctor schedules.

10. REFERENCES

1. **React Official Documentation.** <https://reactjs.org/docs/getting-started.html>
2. **SQLite Documentation.** <https://www.sqlite.org/docs.html>
3. **JWT Authentication Guide.** <https://jwt.io/introduction/>
4. **Microsoft .NET C# Documentation.** <https://docs.microsoft.com/en-us/dotnet/csharp/>

11. CERTIFICATE & ATTENDANCE



Date:08.08.2025

INTERNSHIP COMPLETION CERTIFICATE

This is to certify that

Ms.Jayasri P

has successfully completed a **one-month internship** in the domain of **Full Stack Web Development** from **(8th July 2025 to 8th August 2025)** at **Kosoft IT solutions pvt ltd, Pondicherry.**

During this internship, the intern has worked on technologies including:

- **Frontend:** React.js
- **Backend:** ASP.NET Core, Flask
- **Databases:** MySQL and SQLite

The intern has actively participated in the development of practical web applications, demonstrated excellent learning capability, and completed project tasks with diligence and enthusiasm.

We appreciate their contribution and wish them the very best for their future endeavors.

For KOSOFT IT SOLUTIONS PVT. LTD.,


JAYAKEERTHANAPRIYA
(Director)

Authorized Signatory

Jayakeerthanapriya

Director/ Mobile and IOT lead

Kosoft IT solutions pvt ltd, Pondicherry.

KOSOFT IT SOLUTIONS PVT. LTD.,

Registered Address: No.109, Montorsier Street, Puducherry - 605001 | Functional Address: No. 208, II Floor, 100 Feet Road, Mudaliarpur, Puducherry - 605004
Mobile: 9943588776, 9943588775

ATTENDANCE CERTIFICATE FOR INDUSTRIAL TRAINING / INTERNSHIP

(Attendance Certificate to be signed by the competent authority of the industry mentioning the period of Industrial Training / Internship)

The Student Mr./ Ms. Jayasri. P (Reg.No.) 420422205011
studying Bachelor of Technology programme at Adhiparasakthi Engineering College
in semester 07 at Department of Information Technology has attended Industrial
Training / Internship from 08/07/2025 to 08/08/2025. It is certified that he / she has
completed / not completed the Industrial Training / Internship in our
KOSOFT IT SOLUTIONS PVT. LTD organization / institution.

For KOSOFT IT SOLUTIONS PVT. LTD.,

JAYAKEERTHANAPRIYA

Signature (Director)

Name and Designation of the Officer
Seal of the Organization